

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain the encoder–latent–decoder structure of an autoencoder;
- prepare image data for reconstruction-based learning;
- implement a fully connected autoencoder;
- implement a Convolutional Autoencoder for image reconstruction;
- introduce controlled noise and build a denoising autoencoder;
- evaluate reconstruction using MSE, MAE and SSIM;
- visualize and interpret latent representations;
- explain the probabilistic latent representation of a VAE;
- implement the reparameterization trick;
- calculate and interpret reconstruction and KL-divergence losses;
- generate new images by sampling from a learned latent space; and
- compare deterministic and variational latent representations.

3. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

4. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

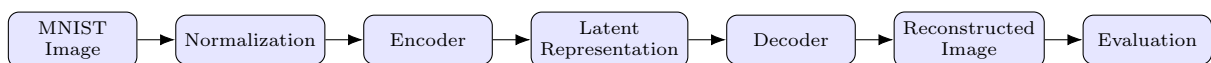
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

5. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

6. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

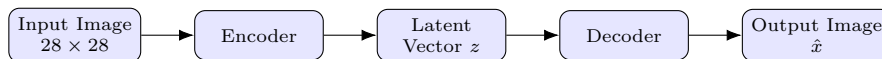
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

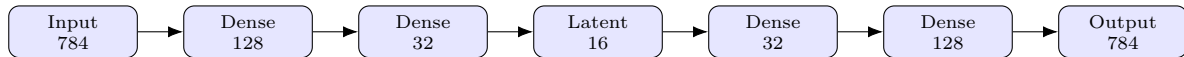
7. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

8. Required Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

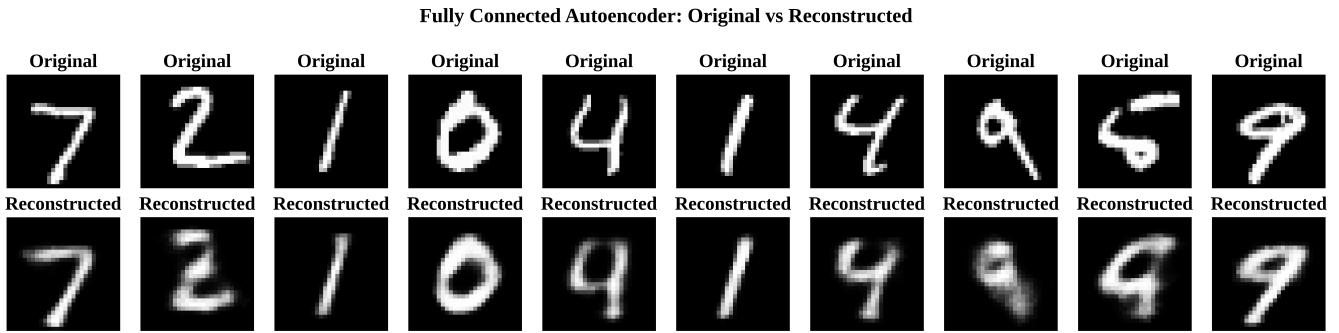


Figure 1: Original vs Reconstructed images produced by Fully Connected Autoencoder

Inference: The image compares original MNIST test digits versus outcomes generated from Fully Connected Autoencoder. The reconstructed data gets the digit's class and orientation right, but shows some blurriness and softness of the strokes. The process of flattening a 2D image into a 1D vector loses spatial locality, pushing the fully connected layer to learn correlations on a more global level which induces low-pass filtering according to the MSE criterion. As we see, the fully connected auto encoder does manage to learn relevant compressed representations, but does not have the spatial inductive bias that is necessary for high-fidelity reconstruction.

Plot 2: Training and Validation Reconstruction Loss

- X-axis: Epoch
- Y-axis: Reconstruction loss
- Curves: Training loss and validation loss

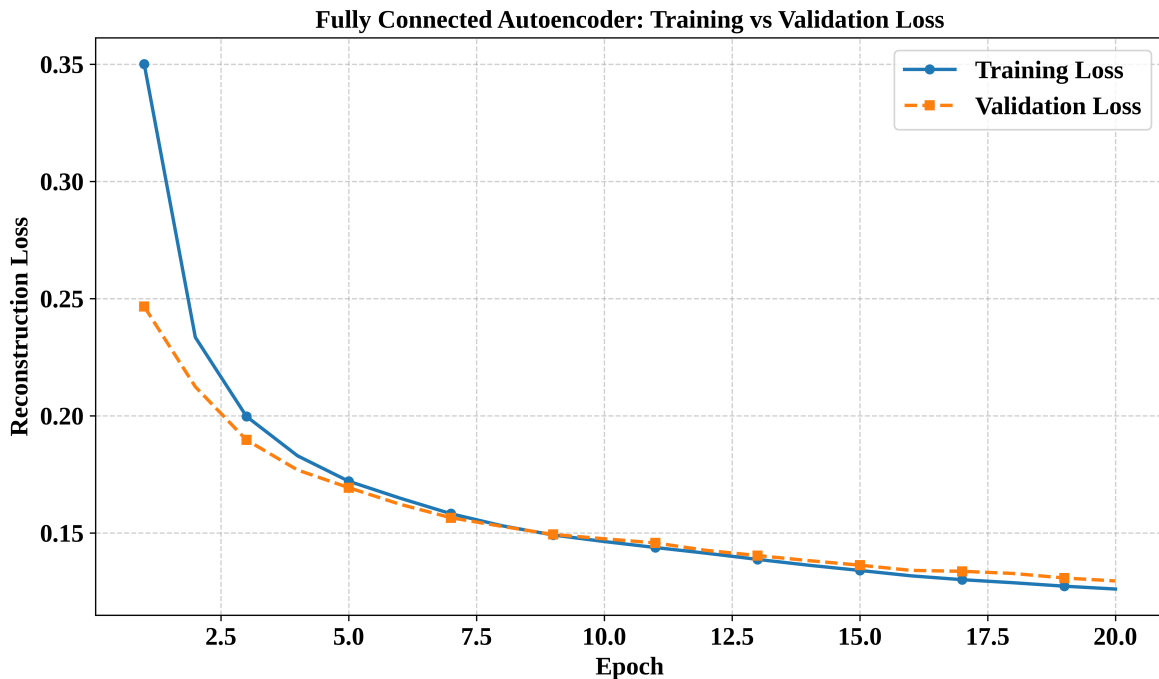


Figure 2: Training and validation reconstruction loss (Binary Cross-Entropy) versus training epochs for the Fully Connected Autoencoder.

Inference: As can be seen from the graph, the Binary Cross-Entropy loss decreases rapidly during the first five epochs, after which there is a slow decrease up to the 17th epoch, after which the curves overlap, which means that there is no overfitting. Due to the limitation imposed by the bottleneck, it becomes impossible for the network to memorize each data point individually, meaning it becomes possible to learn how to reconstruct with the help of the Adam Optimizer.

9. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

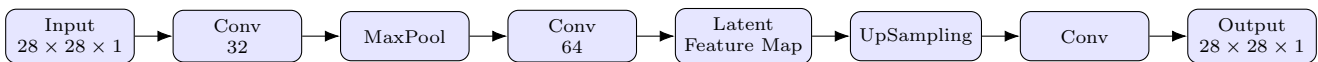
Report:

Metric	Value
Test MSE	0.022779
Test MAE	0.059762
Mean SSIM	0.728950

10. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

11. Comparing Fully Connected and Convolutional Autoencoders

Train both models using the same training and test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022779	0.059762	0.728950	211,040
Convolutional AE	0.002589	0.015037	0.973643	74,497

Plot 3: Reconstruction Comparison

Display:

Original $\rightarrow$ FC-AE Reconstruction $\rightarrow$ CAE Reconstruction.

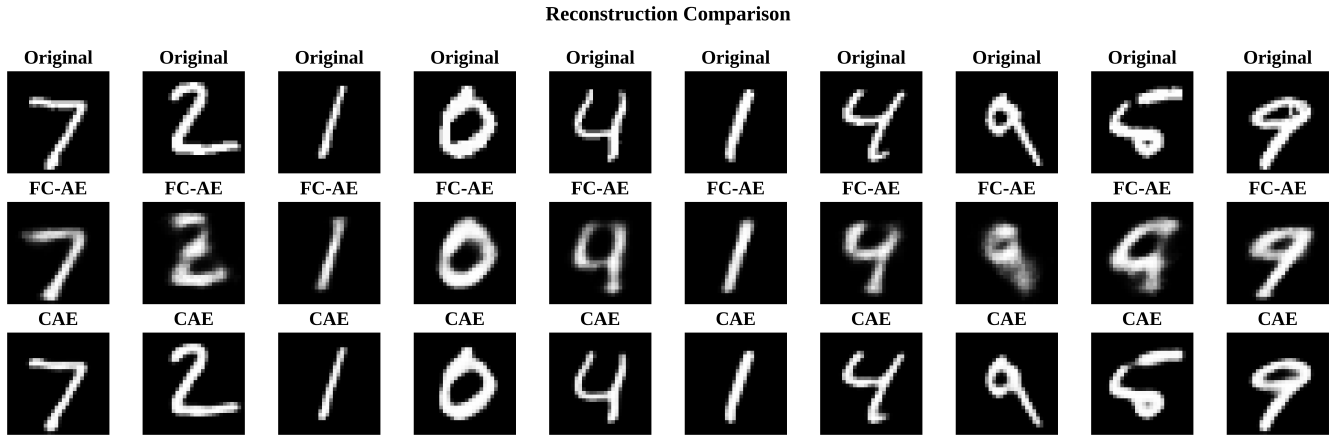


Figure 3: Reconstruction comparison across Original MNIST test digits, Fully Connected AE reconstructions and Convolutional Autoencoder

Inference: Through convolution, it is easier for the model to maintain proximity of adjacent pixels and their features, such as edges and curves. Through the CAE, it can be observed that the digits in the image are better and clearer compared to fully connected auto encoder.

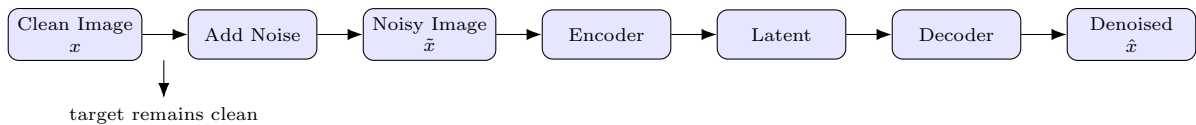
12. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image.

The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



13. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

14. Required Denoising Plots

Plot 4: Clean vs. Noisy vs. Denoised

For at least 10 images, display:

Clean | Noisy | Denoised.

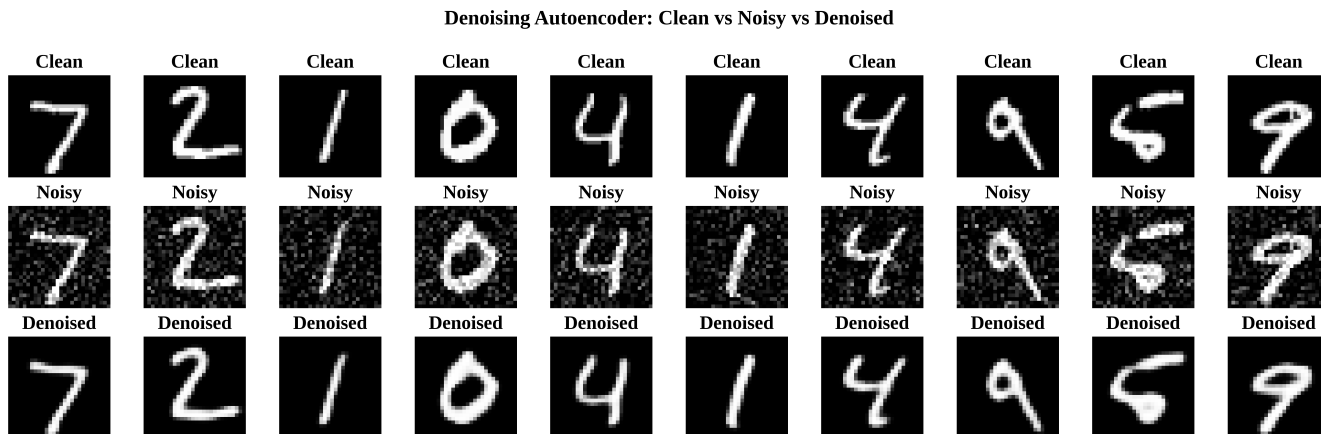


Figure 4: Clean vs. Noisy vs. Denoised images

Inference: It is clear from this that the denoising autoencoder is successful in eliminating noise but also retaining the necessary features in the digits. The Denoising CAE reduces the noise and connects the disconnected strokes by mapping the noisy inputs back onto the clean data manifold. From the image above, it is evident that the denoised images resemble the clean images with smoother and connected strokes.

Plot 5: Noise Level vs. Reconstruction Quality

For each noise level, calculate:

$$MSE, \quad MAE, \quad SSIM.$$

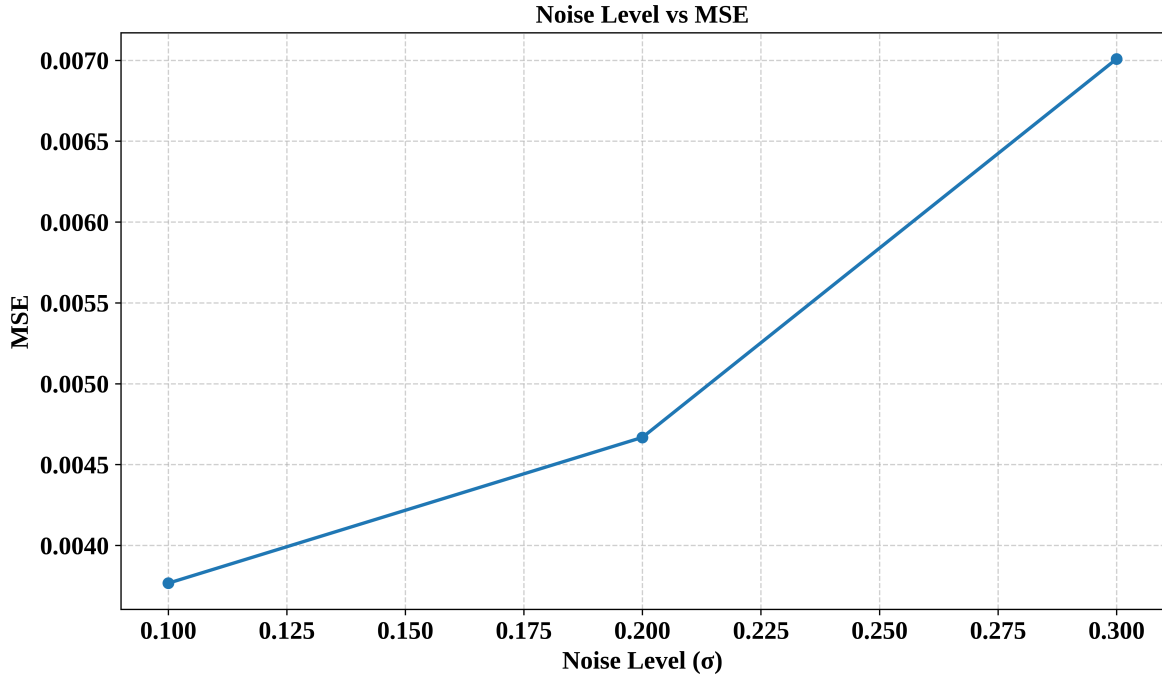


Figure 5: Comparison of MSE of Denoising autoencoder for different noise levels

Inference: As the value of the noise standard deviation increases from $\sigma = 0.1$ to 0.3 , the reconstruction MSE also increases. The slope becomes much steeper after $\sigma = 0.2$. The reason, for this is the nature of the mean squared error. The mean squared error punishes pixel deviations very harshly. The mean squared error punishes pixel deviations very harshly.

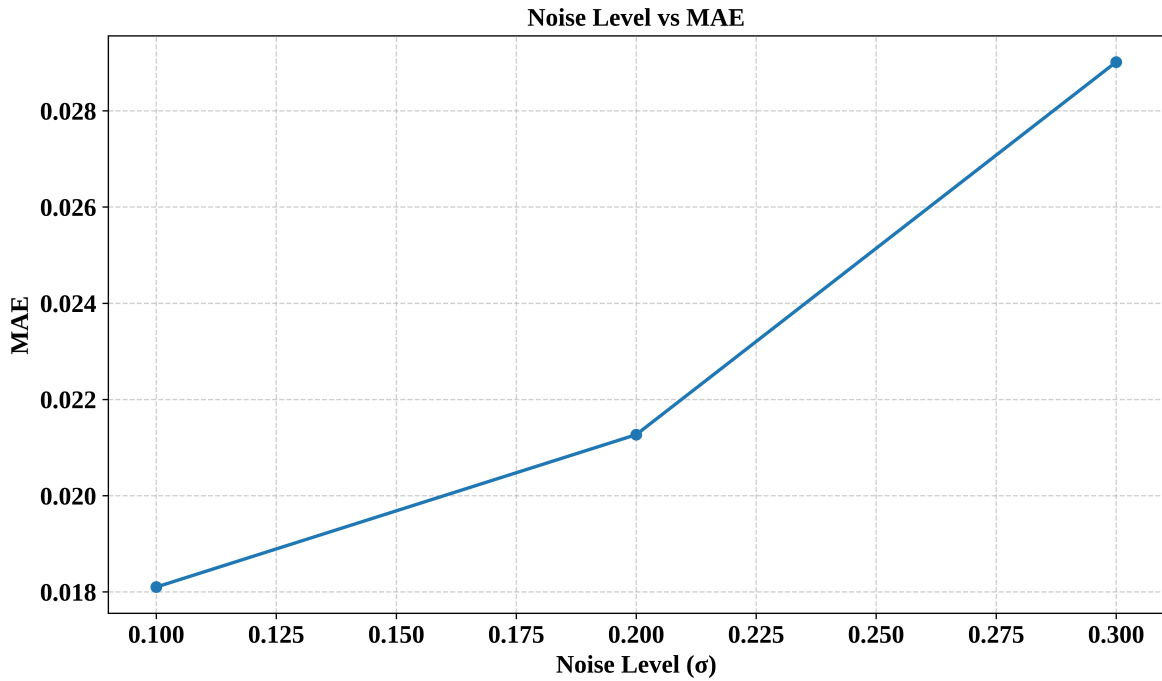


Figure 6: Comparison of MAE of Denoising autoencoder for different noise levels

Inference: The Mean Absolute Error rises steadily from 0.018101 at $\sigma = 0.1$ to 0.029014 at $\sigma = 0.3$. Because MAE

measures residual errors uniformly across all pixels without squaring this steady upward trend confirms a direct and consistent degradation, in pixel-level fidelity and background noise suppression as input corruption grows.

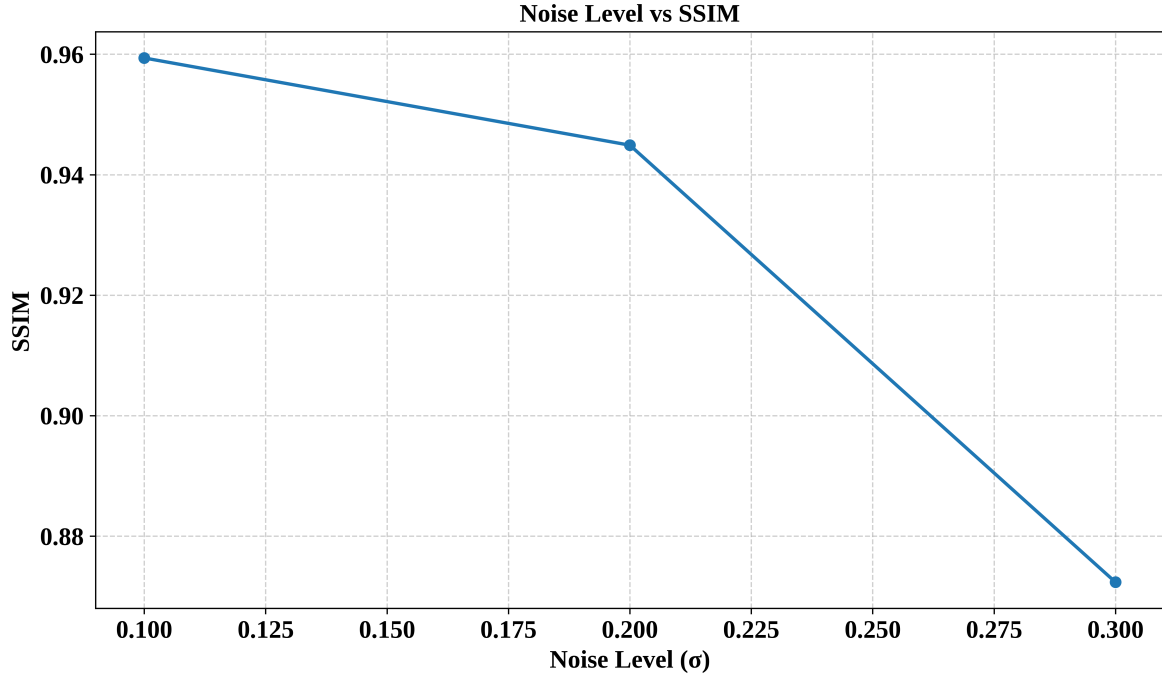


Figure 7: Comparison of SSIM of Denoising autoencoder for different noise levels

Inference: The Structural Similarity Index remains high at noise levels with 0.959378 at $\sigma = 0.1$ and 0.944897, at $\sigma = 0.2$. When the noise rises to $\sigma = 0.3$ The Structural Similarity Index drops noticeably to 0.872354. This shows that convolutional denoising filters keep structures intact when the corruption is mild. However heavy noise really hurts structural coherence, contrast and edge definition.

Report:

Noise Level	MSE	MAE	SSIM
0.1	0.003766	0.018101	0.959378
0.2	0.004668	0.021268	0.944897
0.3	0.007008	0.029014	0.872354

Inference:

The higher the noise level, the greater the MSE and MAE become while the SSIM decreases, which shows that the reconstruction quality is getting worse and worse. The model still retains the main structure of the original image and achieves good denoising even in the case of higher corruption.

15. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

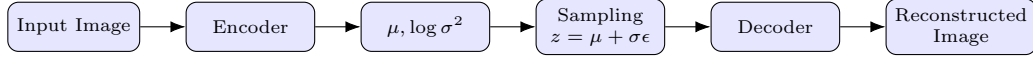
$$q_{\phi}(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



16. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

17. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

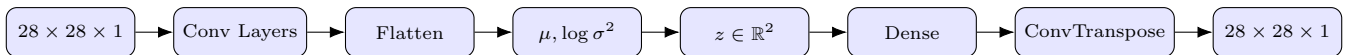
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



18. Exploring the VAE Latent Space

Since the latent dimension is two, students can visualize the encoded test images in a two-dimensional plane.

Plot 6: VAE Latent Space

- X-axis: z_1
- Y-axis: z_2
- Color/marker category: digit label

The labels are used only for visualization; they are not supplied to the VAE during training.

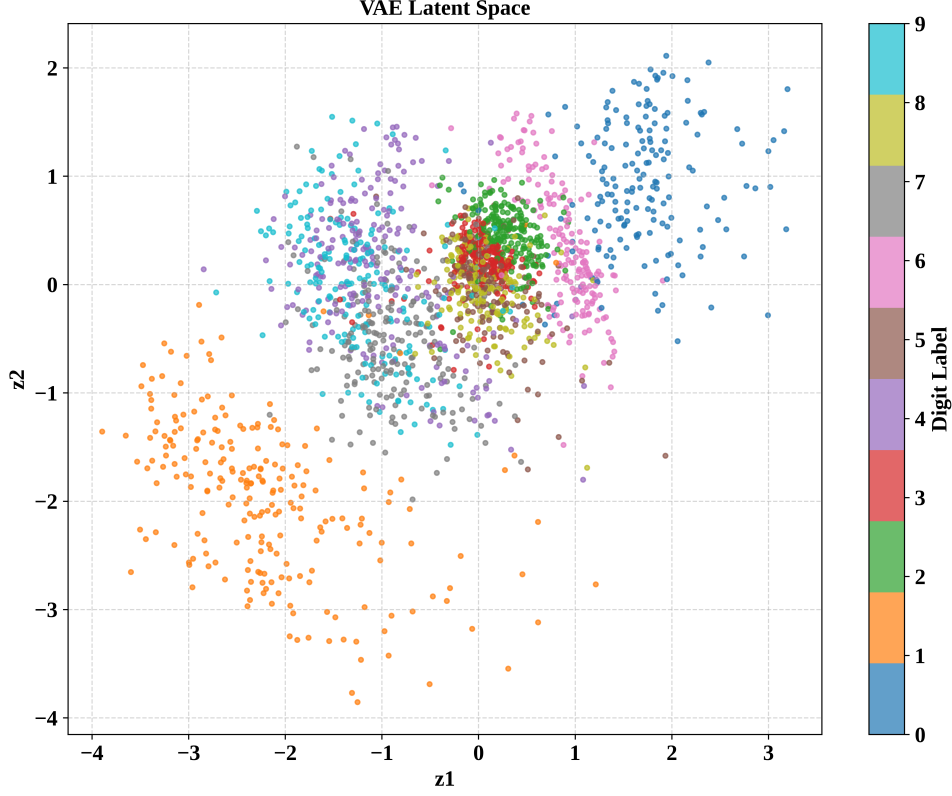


Figure 8: VAE Latent space

Inference: It is clear from the 2D scatter plot that the latent space for the VAE model is compact, smooth, centered around the origin and consists of distinct clusters for digit classes without any form of label guidance. The uniquely structured digits (1, 0, 7) are located in distant areas, while visually similar digits (3, 5, 8, 9) have overlapping clusters. The clustering pattern is a result of the KL divergence constraint which imposes an approximation of $\mathcal{N}(0, I)$ for all the posterior distributions of the encoder network, thus not allowing isolated clusters and creating continuous, hollow-free manifold.

19. Generating New Images with the VAE

Sample points from

$$z \sim \mathcal{N}(0, I).$$

Pass the sampled latent vectors through the decoder.

Plot 7: Randomly Generated Images

Generate at least 25 images using randomly sampled latent vectors.

Display them in a grid.

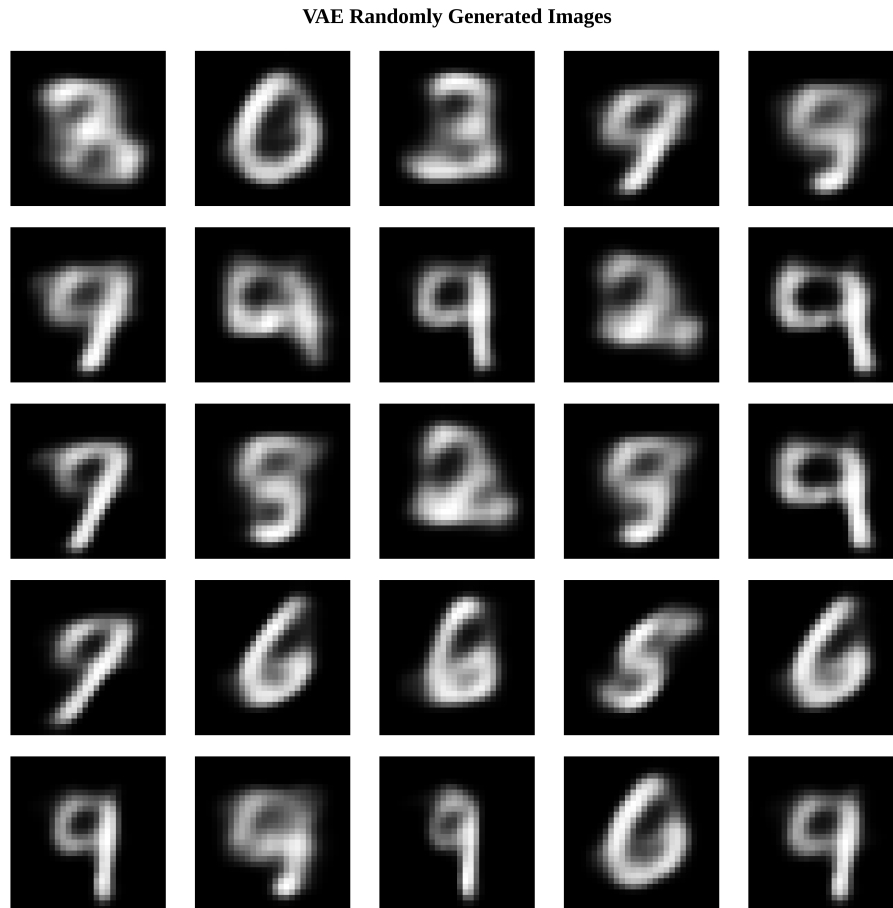


Figure 9: Randomly generated sample images by VAE

Inference: The 5 x 5 grid contains 25 images that have been generated from 25 samples drawn from the VAE prior as $z \sim \mathcal{N}(0, I)$ and then decoded; no input images have been used for this generation process. All but two of the generated images can be identified as handwritten digits such as 0, 1, 6, 7, 8 or 9, which exhibit reasonable variability in terms of stroke thickness and curvatures, indicating that the decoder maps prior probability density on the actual manifold of digits. Some of the images lying at the boundaries between classes look somewhat blurred, which is understandable given the 2D nature of the bottleneck.

20. Latent Space Interpolation

Select two latent vectors:

$$z_A, \quad z_B.$$

Construct

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in [0, 1].$$

Use values such as:

$$\alpha = 0, 0.1, 0.2, \dots, 1.$$

Decode every interpolated vector.

Plot 8: Latent Space Interpolation

Display the decoded images in sequence.

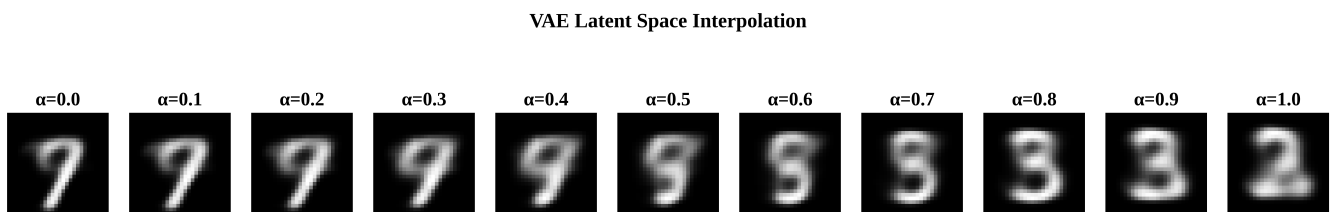


Figure 10: Latent space interpolation of the images of digits given by Variational Autoencoder

Inference: The decoded trajectory across $\alpha \in [0, 1]$ shows a continuous change from digit 7 to intermediate shapes that look like 9, 8 and 3 and finally ends at digit 2. There are no corrupted parts in the transition. This clean flow proves that the VAEs Gaussian prior and KL divergence regularization help create a dense and continuous latent manifold. There are no gaps or sudden jumps, in the latent space. The entire path stays connected and well-behaved

21. Reconstruction Comparison Across Models

Compare the three reconstruction-based models:

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.022779	0.059762	0.728950	211,040
Convolutional AE	0.002589	0.015037	0.973643	74,497
Denoising CAE	0.004679	0.021278	0.944660	74,497

For the VAE, report its reconstruction loss separately because its total objective also contains the KL-divergence term.

VAE Metric	Value
Reconstruction Loss	0.0
KL Loss	0.0
Total Loss	0.0
Test MSE	0.042663
Test MAE	0.099256
Mean SSIM	0.500027

22. Source Code

Source: Github

Github Repository Link:

https://github.com/ssvibitha/DeepLearningLab_2026-27/blob/main/Lab7_Autoencoders/DL_Lab7.ipynb

23. Reconstruction Error Distribution

Calculate the per-image reconstruction error:

$$e_i = \frac{1}{D} \sum_{j=1}^D (x_{ij} - \hat{x}_{ij})^2.$$

Plot 9: Distribution of Reconstruction Errors

Plot a histogram of e_i over the test set.

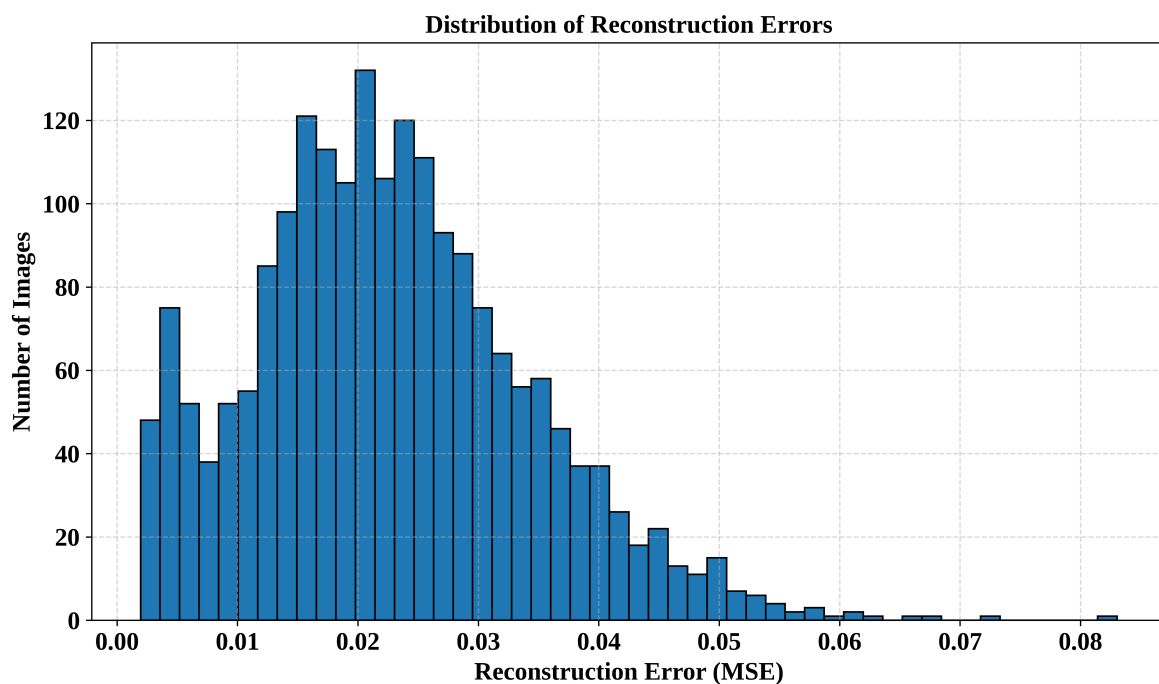


Figure 11: Distribution (histogram) of per-image Mean Squared Reconstruction Errors on the MNIST test set.

Inference: The graph reveals a very right-skewed distribution where the majority of images have low reconstruction errors, and a small number of them have high reconstruction errors. High reconstruction error images typically depict unique handwriting patterns, such as broken characters, highly skewed characters, and unclear digits. This is due to the fact that the autoencoder picks up common patterns more easily than unique ones.

24. Required High-Error Image Analysis

Select the five test images having the largest reconstruction errors.

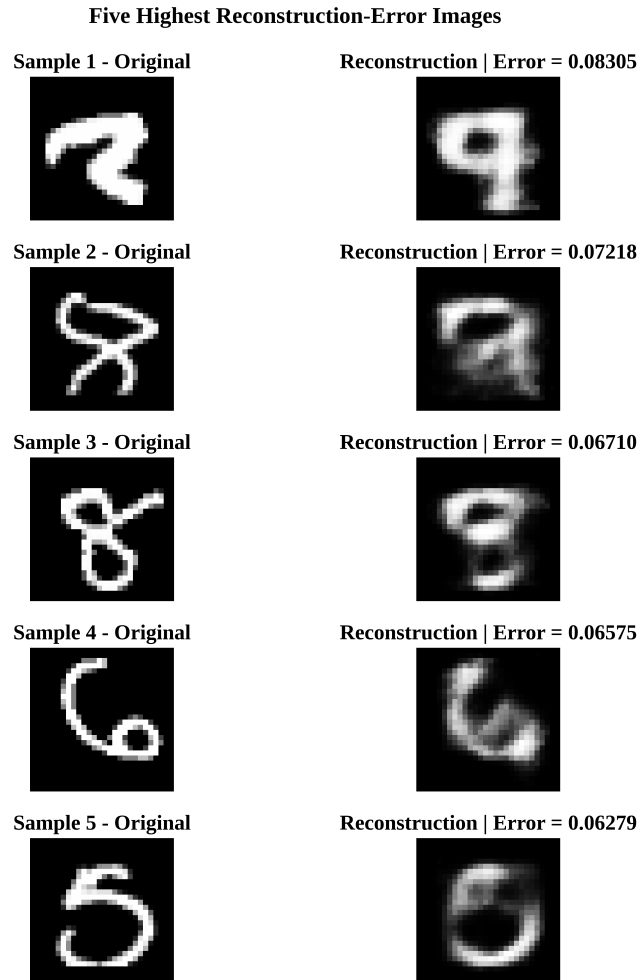


Figure 12: Images having the largest reconstruction errors

Inference: Large values of reconstruction error are found in handwritten images that contain stroke discontinuities, odd slants, faint strokes, or peculiar digit formations. These rare patterns pose more difficulties for the autoencoder to be reconstructed correctly. The model is able to learn common digit patterns better and would often smooth rare patterns to form common shapes. Consequently, reconstruction error can help identify rare patterns of handwriting.

25. Consolidated Results

Model	MSE	MAE	SSIM	Parameters	Time
Fully Connected AE	0.022779	0.059762	0.728950	211,040	22 sec
Convolutional AE	0.002589	0.015037	0.973643	74,497	698 sec
Denoising CAE	0.004679	0.021278	0.944660	74,497	657 sec
VAE	0.042663	0.099256	0.500027	485,957	671 sec

The values must be obtained from the student's own execution.

26. Additional Latent-Dimension Study

Repeat the basic autoencoder using:

$$d_z \in \{2, 8, 16, 32\}.$$

Report:

Latent Dimension	MSE	SSIM	Parameters
2	0.046838	0.434463	210130
8	0.028525	0.665247	210520
16	0.021986	0.738112	211040
32	0.018799	0.771977	212080

Plot 10: Latent Dimension vs. Reconstruction MSE

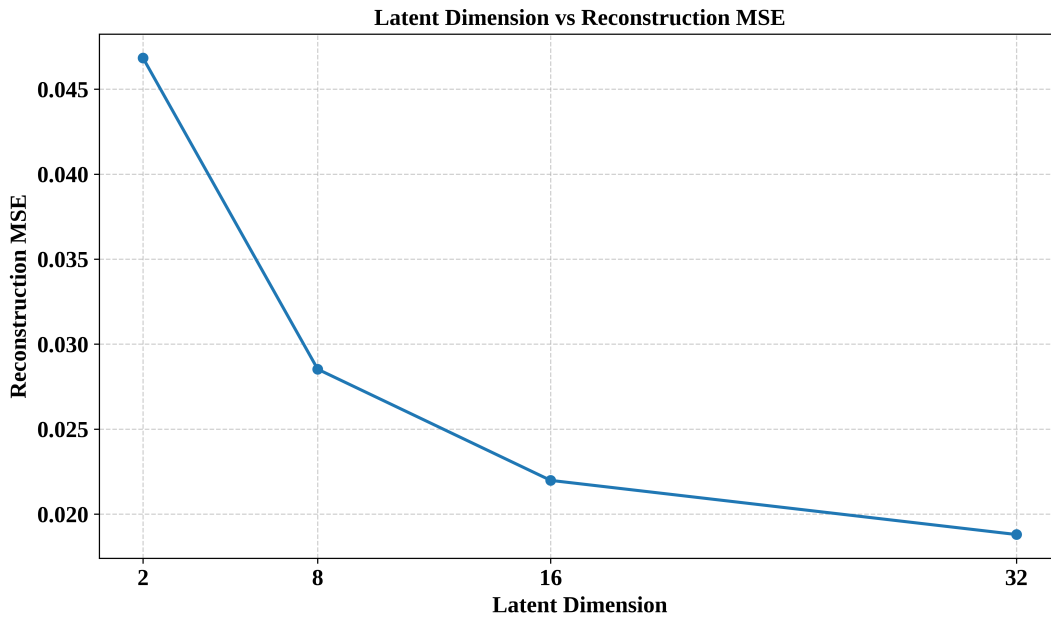


Figure 13: Latent dimension vs. Reconstruction MSE for basic autoencoder

Inference: Changing the latent dimension from $d_z = 2$ to $d_z = 32$ makes the reconstruction MSE fall sharply from 0.046838 to 0.018799. The biggest drop happens between $d_z = 2$ and $d_z = 8$. When the bottleneck is very small, with $d_z = 2$ the latent dimension cannot keep the variations in digit strokes. When the latent dimension is larger the latent dimension gives room to rebuild the fine structural details accurately. This shows the tradeoff: more compression means less reconstruction quality and less compression means better reconstruction.

27. Discussion Questions

1. What is an autoencoder?

Answer: An autoencoder is an unsupervised neural network. It consists of encoder, bottleneck and decoder. The encoder compresses the input into a latent bottleneck and a decoder reconstructs the input. The network is trained by minimizing reconstruction error between the original input and reconstructed output. The bottleneck makes sure the network learns dominant hidden patterns rather than memorizing individual samples.

2. What is the purpose of the encoder?

Answer: The encoder acts as a dimensionality reduction and feature extraction function. It maps high dimensional raw input data into a compact latent vector space. It progressively discards redundant information, noise, and irrelevant variations while preserving essential semantic features. This compressed code captures the most important features of the data needed to reconstruct the original input accurately.

3. What is the purpose of the decoder?

Answer: The decoder is a generative reconstruction network. The primary task of a decoder is to reconstruct the original input in a high-dimensional space from the low-dimensional latent code obtained from the encoder. It synthesizes an approximation of the input using learned structural priors by inverting the bottleneck compression. The decoder ensures that the latent space contains enough information fidelity to reconstruct identifiable high quality images.

4. What is a latent representation?

Answer: The decoder is a generative reconstruction network. The primary task of a decoder is to reconstruct the original input in a high-dimensional space from the low-dimensional latent code obtained from the encoder. It synthesizes an approximation of the input using learned structural priors by inverting the bottleneck compression. The decoder ensures that the latent space contains enough information fidelity to reconstruct identifiable high quality images.

5. Why is the latent representation usually smaller than the input?

Answer: If the latent dimension is less than the input, there arises an information bottleneck, forcing the network to retain only important aspects of the input data and not allowing it to memorize the input data. Thus, it is possible to reconstruct the original input using just the important information and discarding all the noise.

6. What happens when the latent dimension is too small?

Answer: In case the latent dimension is too constrained (for highly dimensional data), an information bottleneck happens. There is not enough expressive power of the network to capture fine structure and high frequency details. The reconstruction error (MSE) is very high, the fine stroke boundaries are lost and there is high blurriness of the reconstructed images. In addition, different classes/ features might be overlapped or even collapsed in latent space.

7. What happens when the latent dimension is increased?

Answer: A larger latent dimension means that there is more capacity for information storage within the neural network, making it possible to extract detailed information such as edges and textures. This tends to improve the quality of reconstructed images and lowers the error. However, an excessively large latent dimension will only make the network copy the input image rather than extracting features. This reduces the effectiveness of data compression.

8. Why can a convolutional autoencoder preserve spatial information more effectively?

Answer: Convolutional Autoencoders (CAEs) retain spatial structure due to their use of local receptive fields, weight sharing, and 2-dimensional activation maps instead of flattening the image into a 1D vector. The flattening of an image in a fully connected neural network causes the loss of the 2-dimensional geometric structure, spatially adjacent nature, and local dependencies between the pixels. CAEs have built-in translation equivariance that enables them to identify local visual elements (edges, curves, loops) irrespective of their location.

9. Differentiate an autoencoder and a denoising autoencoder.

Answer: While a regular autoencoder takes the input vector x and attempts to reproduce the same input vector after putting it through the bottleneck process, a denoising autoencoder (DAE) takes the corrupted input vector $\tilde{x} = x + \eta$ as the input while considering the original input vector x as the target. Hence, the denoising autoencoder will train itself to ignore any kind of noise in the training data and learn the features of interest in the training data.

10. Why is the clean image used as the target in denoising?

Answer: As the clean image serves as the target, the learning process ensures that the network captures the manifold of the noiseless data points instead of trying to reconstruct the noise itself. Thus, the optimization function discourages the presence of any noise in the reconstructed image, encouraging the network to identify the invariant features of the underlying manifold. It results in the learning of an orthogonal projection of the noisy ambient space onto the manifold of dense data points.

11. What happens when the noise level is increased?

Answer: When the variance or standard deviation of input noise (σ) increases, the noisy points move farther away from the clean data manifold. In moderate amounts of noise, the autoencoder is resilient and can filter out noise successfully, although there is some loss in edge sharpness. But when there is too much noise, the important

structural elements get covered up. Therefore, there is a rapid drop in the value of SSIM, an increase in MSE, and blurring in the reconstructed image.

12. Why are MSE, MAE and SSIM useful for image reconstruction?

Answer: MSE and MAE give pixel-by-pixel numbers that show how much the rebuilt image differs from the original. MSE gives penalties for large mistakes. Yet pixel-by-pixel numbers do not always show how a human sees the picture and they can make a blurry picture look good. SSIM fixes this problem, by checking how light, contrast and structure have changed between pixels. Using MSE, MAE and SSIM together gives a test of both the numbers and the way the image looks to people.

13. What is a Variational Autoencoder?

Answer: A Variational Autoencoder (VAE) is a directed generative model that blends deep neural networks with variational Bayesian inference. In a Variational Autoencoder the encoder does not send an input to a point; it sends the input to the parameters of a probability distribution: a mean μ and a log-variance $\log \sigma^2$. The decoder then takes a sample from this distribution. Uses it to rebuild the input. By training, with an objective that balances how well the input is rebuilt and how tightly the latent codes are kept in check a Variational Autoencoder guarantees that the latent space remains smooth and continuous making it easy to generate samples.

14. How does a VAE differ from a conventional autoencoder?

Answer: A conventional autoencoder maps each input image to a point in latent space. This deterministic mapping often creates gaps, breaks and overfitting. By contrast a VAE maps inputs to smooth probability density functions. The VAE uses Kullback-Leibler (KL) divergence to keep the latent space close, to a normal prior. This rule makes the latent space smooth, full and free of holes. While a conventional autoencoder can only reconstruct and compress a VAE can also create believable images by sampling from the prior distribution.

15. Why does a VAE learn a distribution instead of a single latent vector?

Answer: The learning of distributions creates noise and smoothes out some parts of the space. This way, the encoder does not only remember but also places the input data into sensible places. When the inputs are mapped into the regions by mu and sigmoid portions of the hidden space, the outcomes come out to be quite alike. The outputs become unreasonable because of this way. The distribution technique enables us to generate data based on our knowledge of mathematics.

16. What is the reparameterization trick?

The reparameterization trick is a way to recast the algorithm such that the gradients can be passed backwards through random sampling nodes in a neural network model. Direct random sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ cannot be differentiated due to the fact that the random variables do not have continuous gradients with respect to the parameters of the model. Instead, the trick separates the randomness by drawing a random noise vector $\epsilon \sim \mathcal{N}(0, I)$ and calculating $z = \mu + \sigma \odot \epsilon$ deterministically.

17. Why is ϵ sampled from a standard normal distribution?

Answer: ϵ drawn from the standard Gaussian distribution $\mathcal{N}(0, I)$ yields a stochastic source of parameters with mean 0 and variance 1 that is entirely independent of the weights of the model. This independence guarantees that the random number generator does not contribute to the backpropagation of gradients. In addition, scaling and shifting a standard Gaussian by σ and μ respectively results in a Gaussian distribution $\mathcal{N}(\mu, \sigma^2)$, which is precisely what is desired.

18. What is the role of KL divergence in the VAE loss?

Answer: The Kullback-Leibler (KL) divergence is a kind of statistical regularizer for how much the posterior $q_\phi(z|x)$ produced by the encoder deviates from the prior $p(z) = \mathcal{N}(0, I)$. In other words, it penalizes the cases where the distributions have the means not close to zero and variance not equal to one. This way, the model won't have separated regions of different classes in the latent space. The latent space will be smooth, centered, and densely occupied.

19. What is the prior distribution used in the VAE?

Answer: The usual prior used for VAE is that of an isotropic multivariate Gaussian having the probability density function of $p(z) = \mathcal{N}(0, I)$. This particular prior defines the latent variables to be zero-mean, independent of each other, and unit-variance in all dimensions. This prior provides a simple and analytical prior for determining the closed-form KL-divergence for posterior Gaussians. In addition, the normality of the prior in standard form

helps in sample generation.

20. **Why is a two-dimensional latent space useful for visualization?**

Answer: A two-dimensional latent space ($d_z = 2$) allows for simple visualization of the latent coordinate system by plotting the latent coordinates on a two-dimensional plane without further dimensionality reduction steps such as PCA or t-SNE. This not only facilitates the interpretation of class separability and manifold structure, but also provides an intuitive means to analyze variations of digit shapes across the latent space by exploring it on a two-dimensional grid. In short, a two-dimensional latent space is an easy-to-interpret probe into the representational geometry of the latent space.

21. **What does latent-space interpolation demonstrate?**

Answer: The latent-space interpolation works by exploring an arbitrary continuous trajectory (e.g., a straight line $z(\alpha) = (1 - \alpha)z_A + \alpha z_B$) in the latent space and decoding the obtained latent vectors as images. This procedure qualitatively confirms that the latent space is continuous, compact, and ordered in a semantically meaningful way. In particular, if the decoded images change in a non-garbled way from one digit class to another, this implies that the latent manifold does not possess holes or gaps. This is a critical property that facilitates decoder learning of the true underlying data distribution.

22. **Why can a VAE generate new images?**

Answer: A VAE is able to generate novel images because its training objective regularizes the encoder distribution to be close to a known, tractable prior $p(z) = \mathcal{N}(0, I)$. Because the latent space is continuous and dense, almost every latent vector drawn from this distribution will correspond to a valid point on the data manifold. To synthesize a novel image, one just draws a random sample $z \sim \mathcal{N}(0, I)$ and passes it through the trained decoder, which will reconstruct a realistic synthetic image with the characteristic features of the training domain.

23. **Why might some VAE-generated images be unclear?**

Answer: The images generated by VAE tend to be blurry because the pixel-wise reconstruction loss (e.g., MSE) penalizes the network for global errors and encourages it to learn the conditional expectation of the distribution. Furthermore, the KL divergence term in the loss function forces the distributions in the latent space to overlap, resulting in a trade-off between reconstruction quality and latent space regularity. Generative capacity being constrained or the decoder trying to interpolate between different modes in the latent space results in the learned model outputting a blurred version of any multi-modal distribution in the training set.

24. **What is the difference between reconstruction and generation?**

Answer: Reconstruction is a process where an existing input image x is turned into a latent space representation and then rebuilt as $\hat{x}$ equal to x . The purpose is to make sure that there is little information lost and that the result is very close to the original image. On the hand generation is when completely new data is created from nothing by taking random latent vectors z from a certain distribution and not looking at any specific input image. While reconstruction checks how well the model can encode and decode generation checks how well the model can create samples, from the distribution it has learned.

25. **Why should test images remain unseen during training?**

Answer: I think that keeping test images hidden during training is essential for determining how the model will perform on new data. If test images were visible to the model during training the model could remember pixel patterns of learning the real shape of the data. Testing the model on test images shows that the autoencoder has learned good general representations instead of just copying specific training examples. It also gives a view of how well the model can reconstruct images in the real world and how reliable it is, at detecting unusual pictures.

28. Additional Exercises

1. Train the convolutional autoencoder with latent dimensions of 4, 8, 16 and 32 and compare reconstruction quality.
2. Compare Gaussian noise and salt-and-pepper noise using the same denoising architecture.
3. Train the denoising autoencoder using two different noise levels and compare SSIM.
4. Replace the convolutional decoder with transposed convolutions and compare the reconstructions.
5. Implement a VAE with latent dimension 2 and 8 and compare the latent representations.
6. Investigate the effect of increasing the KL-loss contribution.
7. Generate a larger grid of VAE samples and discuss diversity.
8. Select two digit regions in the latent space and perform interpolation.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete image-representation and reconstruction pipeline:

$\text{Image} \rightarrow \text{Encoder} \rightarrow \text{Latent Representation} \rightarrow \text{Decoder} \rightarrow \text{Reconstructed Image}$

They should also demonstrate denoising:

$\text{Clean Image} \rightarrow \text{Noise} \rightarrow \text{Denoising Autoencoder} \rightarrow \text{Recovered Image}$

and variational generation:

$\text{Image} \rightarrow (\mu, \sigma) \rightarrow z \rightarrow \text{Decoder} \rightarrow \text{Generated/Reconstructed Image}$

The final report should contain both quantitative evaluation and visual analysis. Numerical performance values must be obtained from the student's own execution.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Diederik P. Kingma and Max Welling, "Auto-Encoding Variational Bayes," ICLR, 2014.
3. Pascal Vincent et al., "Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion," Journal of Machine Learning Research, 2010.
4. Yann LeCun, Corinna Cortes and Christopher J. C. Burges, "MNIST Handwritten Digit Database."
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>
7. scikit-image Documentation: <https://scikit-image.org>